

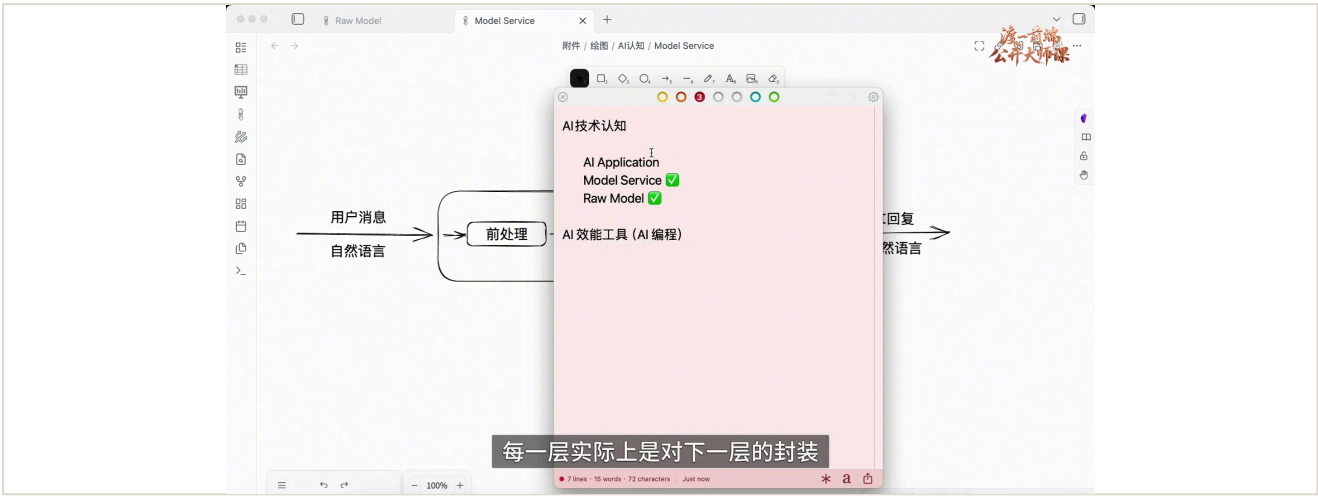
AI 的本质

从 Raw Model、Model Service 到 AI Application，沿着真实请求追踪提示词、技能与工具调用。保留课堂推理、案例及操作细节。

学习路径

- 01 从裸模型、模型服务走到 AI 应用
 - 02 应用与模型服务的边界为什么会移动
 - 03 微调案例：训练时间之外，还有验证成本
 - 04 AI 应用层：应用开发与工具使用
 - 05 课程体系说明与工具使用的定位
 - 06 先理解概念，再学习具体编程工具
 - 07 理解新概念，只盯住输入与输出
 - 08 用前处理、后处理检查这两个问题
 - 09 HTTP 类比：看消息、格式与后续动作
 - 10 搭一层代理，观察请求和响应
 - 11 配置 .env，用 curl 验证代理日志
 - 12 让 Claude Code 通过代理访问模型
 - 13 还没输入问题，应用已经发送了工具描述
 - 14 “你是谁”的答案，来自应用添加的提示词
 - 15 触发一次技能，再去查真实请求
 - 16 SSE 流里的自然语言与工具调用 JSON
 - 17 第一轮：封装输入，模型决定是否调用技能
 - 18 第二轮：读取技能全文，继续调用模型
 - 19 把观察方法迁移到下一个新概念
 - 20 抓住核心与后续课程安排
-

01 · 从裸模型、模型服务走到 AI 应用



课堂画面 · 点击放大

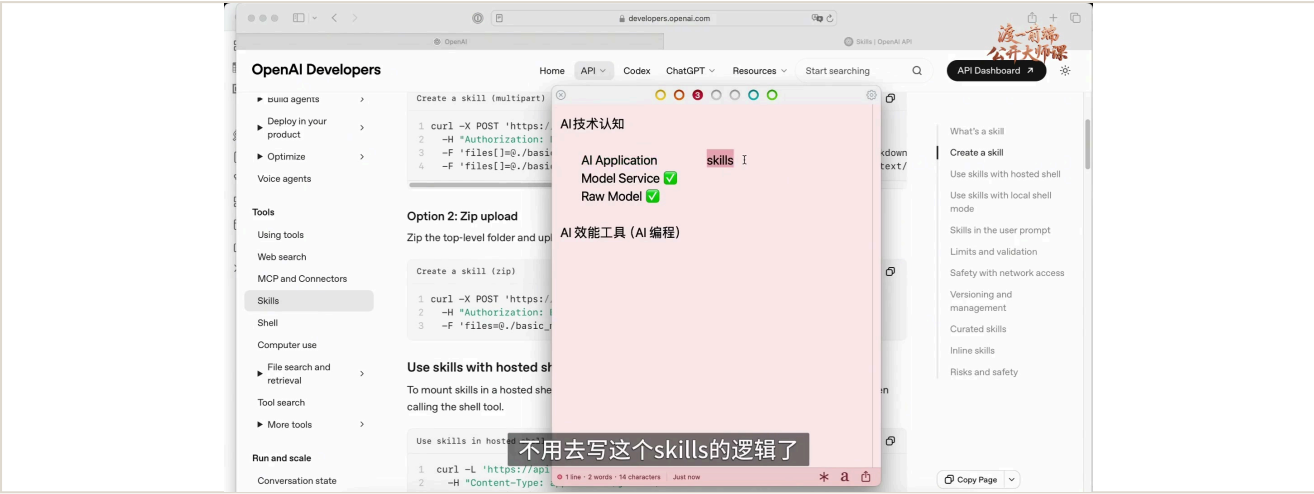
那我是如何看待这些新概念的？我又是如何去学习它的？我要把这个方法教给同学们。所以这节课，你知道为什么这么重要了？你只要掌握好了这个方法，将来无论遇到什么样的概念，随便来，我保证你几分钟时间就搞定。有这么神奇吗？那你听一听就知道了。好，各位同学，这节课的内容，非常非常重要。可以这么说，这节课的内容呢，只要AI存在一天

这节课的内容就会发挥作用一天。我们回顾一下之前，我们讲过了这个裸模型和模型服务，是吧？那么接下来呢，我们要讲这个AI Application，就这一层。我们知道每一层实际上是对下一层的封装，比方说这个模型服务，它是封装了什么呢？裸模型的调用，而我们的AI Application呢，它是封装了模型服务。

画面与校读说明

画面列出 AI Application、Model Service、Raw Model 三层，后两层已经打勾；背景是模型服务的前处理、自回归和后处理示意图。

02 · 应用与模型服务的边界为什么会移动



课堂画面 · 点击放大

它是这么一个关系。然后在讲之前呢，有几点需要给同学们声明一下：第一个呢，就是这两层，他们的分界线，是没那么清晰的。它不像那个模型服务跟那个裸模型之间，它的分界非常清晰。裸模型是这个玩意儿，对吧？得到一个Token列表，然后得到一个概率分布。模型服务呢，用一个自回归，之前讲过的，去调用那个裸模型的功能，然后对它进行前处理和后处理。它界限比较分明。但是这个AI Application呢，和这个模型服务，它们之间的界限没有那么分明。

有很多概念呢，它可能今天在这一层，明天它变到这一层了。比方说举个例子，像同学们现在都用的是那个什么Skills，对不对？，像这个概念呢，最早的时候是在这里，是这一层玩出来的概念。后来觉得这个概念还挺好，然后呢，模型服务商呢，他就把这个Skills这个概念呢，直接就融到这个模型服务层了。也就是说，你这个开发AI应用的，你就不用去管什么Skills了，我这里已经给你做了。比方说咱们来看一下OpenAI的一个接口。

好，你看OpenAI的接口里边，说是有这个skills，也就是你直接调那个API接口，你就可以实现一个skills。也就是说，如果说你要开发AI应用的话，你又调的是OpenAI的接口的话，那么你就在这里边不用去写这个skills的逻辑了，懂我的意思吧？，这只是举了个例子哈。包括像之前的什么MCP什么tools，对吧？那么现在呢，很多模型服务商呢，都直接就支持MCP和tools的

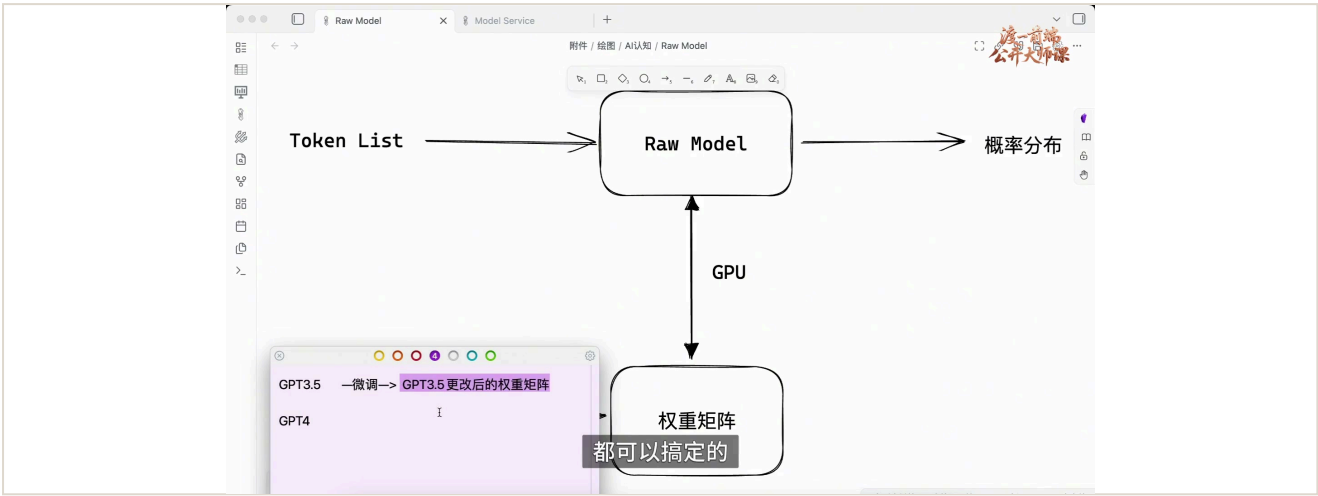
你在开发AI应用的时候呢，就不用去自己处理这一块的功能逻辑了，懂意思吧？就这两层，它的界限没有那么分明。但是呢，很多事儿吧，总得要做，比如Skills这一块跟模型之间的交互逻辑。那么就这一块事儿呢，就看谁做的问题。如果说你使用的模型服务商他没有做的话，那你你就必须要做。反之，如果他做了，你就不需要做了，就这么简单。总归有一个层要做。你模型服务商多做一点呢，那我就可以少做一点。

你少做的话，我就必须要多做，总归有个人需要来做。所以说这两层呢是不太稳定的。我们学到目前，同学可以发现，像这个裸模型这一层，它是非常稳定的。虽然它的训练方式啊、性能啊、参数，里边的一些什么注意力机制啊这些具体的做法呢，可能在不断的更新变化，但它的调用方式是非常稳定的。而到了上一层模型服务和AI应用这一层，是最不稳定的。同学看到那个自媒体上天天上蹿下跳的、大呼小叫的，也就是对这一两层的变动。

画面与校读说明

画面同时展示 OpenAI 文档页面与三层笔记，Skills 被用作功能从应用层进入服务层的例子。这是课堂录制时的接口示例，正文保留讲者的概括，不作为当前所有 API 的能力说明。

03 · 微调案例：训练时间之外，还有验证成本



课堂画面 · 点击放大

能懂这意思吧？其实这也符合咱们技术里边的一个规律：任何一个技术栈，越底层越稳定，越上层越不稳定。就说到这一点，我顺便给大家说一下，这也是为什么现在几乎是没有人去玩这个微调了。我们之前说过，微调、微调和训练其实都是在生成什么，生成这个裸模型要调用的权重矩阵，都是改动这里边的参数。为什么现在没有人去玩微调这个事儿呢？因为从成本上就划不来

比方说哈，举个例子，你这儿的这个权重矩阵，是基于GPT-3.5的。哎，你觉得3.5的功能还不够啊？至少对你们目前公司里边的很多知识呢，是没有经过训练的。你们是公司里面有大量的内部知识。那么过去的做法会怎么样呢？过去的做法会对它进行微调，是吧？微调的作用是什么？就是把它原本生成的权重矩阵呢，某一些参数给它改了

都要经过很多的二次训练的哈。微调过后呢，是不是生成一个新的啊？就基于GPT-3.5的权重矩阵，就更改后的权重矩阵，这个没问题吧？好，这事儿儿要做多久呢？少说你要两三个月。微调其实本身呢，它训练花不了多少时间，主要是要验证它训练的结果是不是符合预期的。这个验证过程是非常麻烦的，你整了两三个月

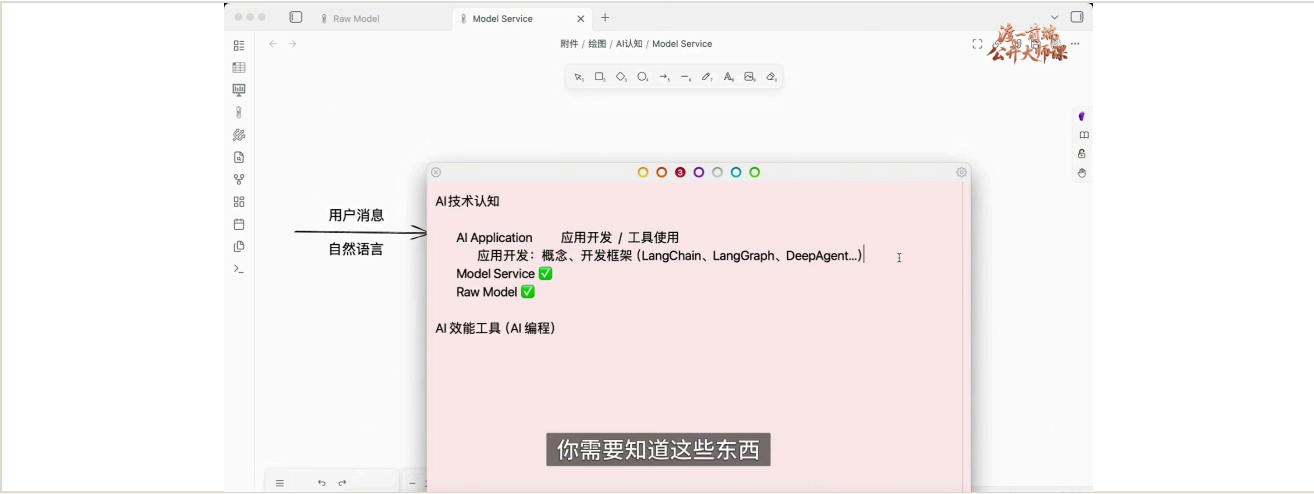
这是至少，我往少了说，你搞不好的话，半年都是有可能的。你好不容易整了半年，把它微调出来了，训练效果还不错。好，结果GPT-4出来了，你会发现GPT-4，你不用微调，你就给它喂一些提示词，它的上下文窗口更大，识别更准确，幻觉更少。你会发现，即便不用微调，都比你这个微调的结果好。你就给它喂一些提示词，都可以搞定的。所以说，你之前那半年在干嘛？这瞎折腾，你花了那么多成本。

我都不是说算力的成本了。算力其实花不了多少成本，主要是人力的成本。你要大量的去验证这些成本花了过后，你会发现大模型更新呢，比你之前要更强了。你直接用它，比你之前微调的结果更好。所以反复这样折腾下去，有些公司就发现这个微调这个活是真不能干了。因为现在AI发展呢，还没有那么成熟，还在不断的发展过程当中。虽然这个调用方式是稳定的，但是这一块训练的结果不稳定。

画面与校读说明

Raw Model 图中，Token List 指向模型，再输出概率分布；权重矩阵通过 GPU 与模型相连，下方有“训练/微调”箭头。便签比较 GPT-3.5、微调后的矩阵与 GPT-4。微调是否划算、所需两三个月或半年，是本课举例与讲者判断。

04 · AI 应用层：应用开发与工具使用



课堂画面 · 点击放大

所以说去微调呢，划不来。因此现在基本上，除了一些少部分极端的场景之外，没有人去玩微调了。因此现在这玩啥呢？主要就是玩这两层，就是这两层。那这一层你玩不了，就是别人模型服务商给你提供的接口，对吧？你玩不了，那么玩啥呢？就玩这一层。那么这一层无非就两个层面可以玩，一个是啥呢？一个是应用开发。就是我们说的AI应用开发，指的是什么呢？指的就是去调用模型服务商给你提供的接口

然后去做一个具体有价值的AI工具，比如说像我们这个AI Coding，对吧？我们后边要讲的AI效能工具，主要是讲AI编程。那么AI编程的工具，哎，请问大家属于哪一层？属于哪一层？是这一层吗？不是吧？这一层吗？也不是吧？是，是AI应用这一层。好，这一层除了涉及到应用开发之外呢，另外呢还涉及到另一个，就是工具使用。这两个很好理解吧？应用开发什么意思？就是你开发一个AI应用，去调用这些接口，去完成一个AI应用开发。

讲到这，我就顺便多讲一点吧。一个是应用开发层面，它主要是调用这个接口去完成应用开发。那么这里呢，需要知道的是，一个是概念，这里有很多概念，什么tools啊、MCP啊、RAG啊、skills啊等等等一系列的概念。像我们之前说的，很多很多的AI概念，都是跟应用层相关的。当然，我刚才也说了，这个概念呢，它在这两层之间呢是分界线，没有那么明确。有些概念呢，现在已经被模型服务商所吸收了，他自己就给你提供这样的一个东西了。

但是他提供了，你还需要知道，不然的话你使用不知道怎么去使用。一个是你需要知道概念，第二个呢，就是一些开发框架，像什么LangChain、LangGraph，还有什么DeepAgent啊等等等等，一些常见的一些开发框架。这是你需要了解的，就是你如果说要做这个应用开发的话，你需要知道这些东西。好，这些东西呢，就在咱们的架构课里边，给大家看一下。好，你看，咱们架构课里边除了很多的架构项目之外，那么还包含了AI的应用开发相关的东西，这是AI的相关概念。

画面与校读说明

便签把应用层分为“应用开发 / 工具使用”，并列 LangChain、LangGraph、DeepAgent。开发框架名称根据画面校正。

05 · 课程体系说明与工具使用的定位



课堂画面 · 点击放大

然后下边呢是一个LangChain，还有LangGraph，后边还有AI的项目，这个就相当于你们用的什么AI编程，我给你实践了一遍，怎么去做一个AI编程出来，后边还会加入很多的其他的AI应用开发的项目。整个架构课呢，其实目的非常明确，就是如果说你工作了两三年了，然后薪资呢涨不动了，那么这个时候呢，你就需要咱们的整个架构课去提升你的薪资。整个这个架构课，就是说白了，目的很明确，就是打开你一个薪资的空间。

同学们可以来了解一下咱们的课程。高薪课呢，是从零基础到高级业务开发的；然后架构课呢，是从高级业务开发到架构师的AI应用开发的内容。在架构课里边，除了这个体系化课程之外呢，我们还有很多的福利课程。这些福利课程讲的就是一些核心知识点，来自于咱们的付费课，送给大家。他可以在极短的时间里边解决你的一些核心问题。就有些同学呢，特别是白嫖的，就仅仅通过咱们的福利课程也可以完成薪资大幅度的提升。像这些课程，都可以来找咱们去了解了解

找到咱们的方式啊！在咱们账号主页面点击头像进入账号主页，根据提示来寻找我们就可以了。好，话说回来，AI应用这个层面呢，除了应用开发之外，还有一个什么呢？就是我们这门课要讲的就是AI效能工具，最后要讲的，工具使用。那这一块就不涉及到开发了，就是如何来使用工具来提升我们的工作效率。那这里边也会涉及到概念，而且这个概念的认识，和前面概念的认识是一样的，没有什么区别。

画面与校读说明

画面展示课程目录以及高薪课、架构课、兴趣课和福利课程的介绍。正文保留原课的课程介绍与薪资表述，不将其改写为效果保证。

06 · 先理解概念，再学习具体编程工具



课堂画面 · 点击放大

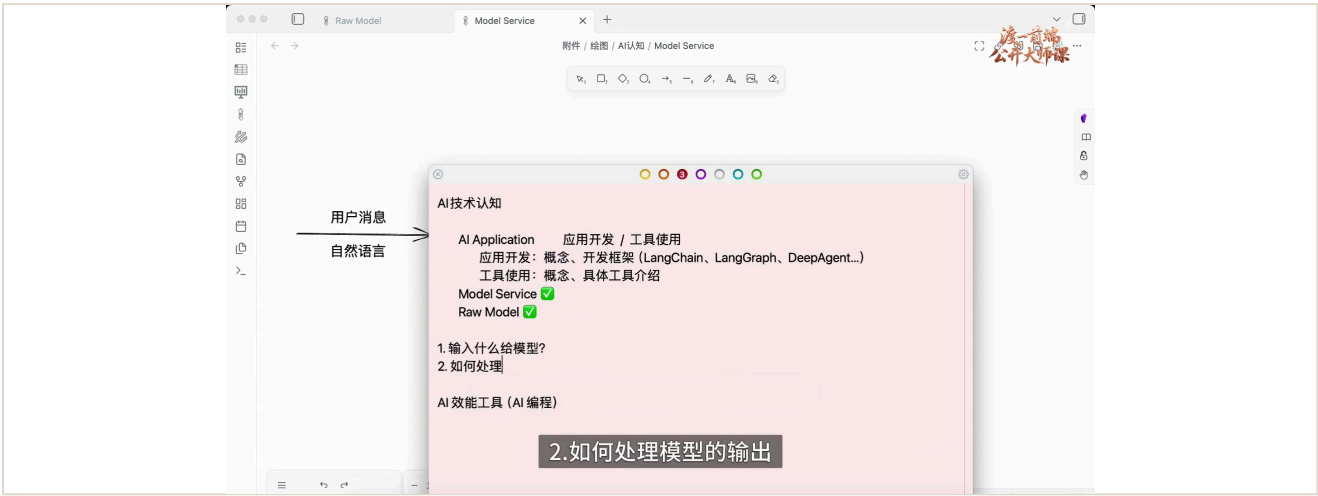
你不认识那些概念，你工具也使用不好。然后才是具体工具介绍，什么Claude Code呀、Codex啊、OpenCode呀这些乱七八糟东西。其实呢，到时候你只要理解了概念过后，我稍微一点拨，你就明白了。很多同学用这个工具用着恼火，不知道该怎么用，就是因为概念不明。好，这个体系我就讲清楚了。我们这门课讲的是什么？前面技术认知讲完过后，我们后面就会讲这个编程工具，就是AI编程工具，主要就是AI编程工具。因为我们这个课呢，面向的是程序员吧，对吧？

你要说什么？AI 短剧也不在我们这个课的范畴之内。那也倒是也属于工具，使用层面，但是不在我们这个课的范畴之内。主要就是AI编程工具。那后边如果说还有一些新奇的工具呢，我们可以介绍介绍，像什么OpenClaw〔名称校读存疑〕啊之类的，对吧？最近比较火。但是很多新奇的工具吧，它的生命周期很短，大部分工具都是玩概念的，生命周期有的可能连一周都撑不过，有些呢能够撑几个月，但是很快就销声匿迹了。不过同学们如果说感兴趣的话，跟大家介绍介绍也没事儿。

画面与校读说明

画面显示应用开发与工具使用的分工，字幕明确列出 Claude Code、Codex、OpenCode。另一个工具在音频中被识别成 OpenCL / Open Cloud，正文暂作 OpenClaw 并保留存疑标记。

07 · 理解新概念，只盯住输入与输出



课堂画面 · 点击放大

好，那么我们这节课干嘛呢？我们这节课是要讲这些概念吗？其实不是，我们这节课要讲的东西比概念更重要。听好了，我们这节课要讲的是如何去理解概念。我这节课给的不是鱼，也不是钓鱼竿，也不是给你讲钓鱼装备。我给的是钓鱼的方法。因为AI目前啊发展太快了，你看，自媒体天天上蹿下跳的，那么它每天都有一个新东西，不搞出来大新闻，他睡不着觉的。每天都要颠覆一下全人类，对吧？

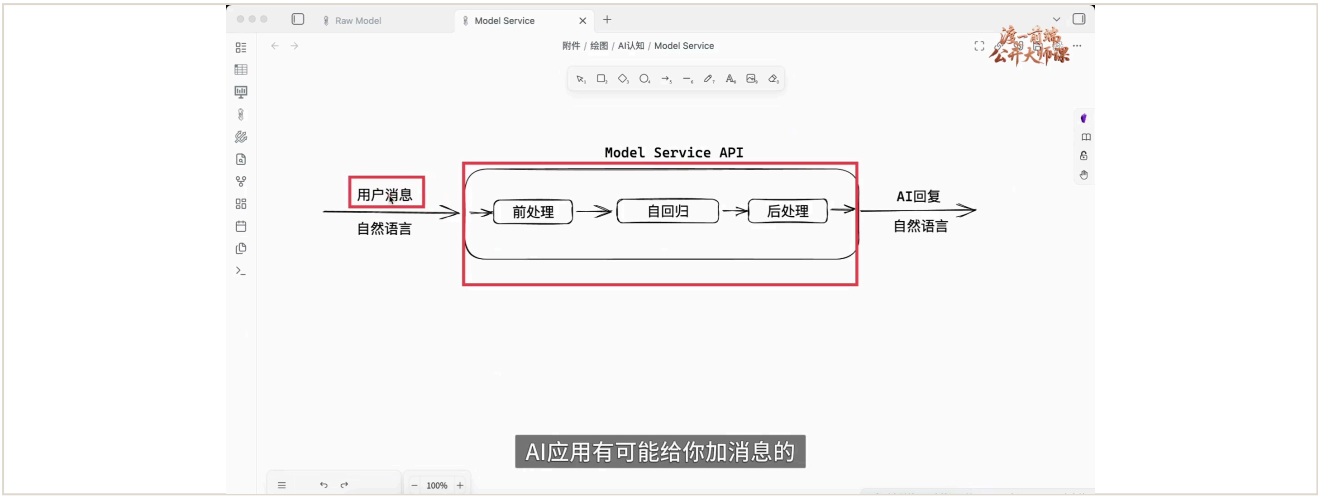
这东西太多了，新的概念层出不穷。我可能这一块刚讲完，可能有些概念就失效了，又不玩了。然后有些新的概念又出来了。那么面对这种情况，同学们该如何自处？因为为什么你们天天的慌？袁老师不慌。因为在我的眼中都一样，新瓶装旧酒没有什么区别。也不能说没什么区别吧，至少本质上没什么区别。严格来说，那我是如何看待这些新概念的？我又是如何去学习他的？我要把这个方法教给同学们。所以这节课，你知道为什么这么重要了把？

你只要掌握好了这个方法，将来无论遇到什么样的概念，随便来，我保证你几分钟时间就搞定的。有这么神奇吗？那你听一听就知道了。好，那咱们开始。首先我给大家抛出一个论点，听好了，这个论点是这么说的啊：所有在裸模型基础上去玩的所有东西，它只能玩两个东西：一、输入什么给模型；二、如何处理模型的输出。无论是模型服务商，还是你的AI应用，因为我们之前说过，这两层呢，它交界呢，并没有那么明显，对吧？你做多一点，我就做少一点。玩的就这两件事儿。第一个，我给模型啥；第二个，他给我一个东西，我怎么办？就这两件事儿，没有别的事儿了。他又玩不出别的事儿了。这是由模型的本质决定的。模型的本质就是输入输出。你给我一个东西，我给你一个东西，中间是经过一串数学运算。

画面与校读说明

便签写出两个问题：“1. 输入什么给模型？2. 如何处理模型的输出？”与这两个问题无关的 ASR 异常尾句已经通过同一时段的局部音频重识别排除，原始块仍保留供核查。

08 · 用前处理、后处理检查这两个问题



课堂画面 · 点击放大

由于模型的本质是这样的，所以我们只能玩这两个东西：我给它啥，拿到它的结果过后，我又做啥。我们来看一下这张图，这是我们之前的模型服务商那个图，就模型服务，他在做什么？你看，我们的前处理的目的是什么？回忆一下，我们前处理要做什么？Token 化，还要给它喂一些系统提示词，对吧？做来去做，是不是目的就是要扔给模型一个东西啊？

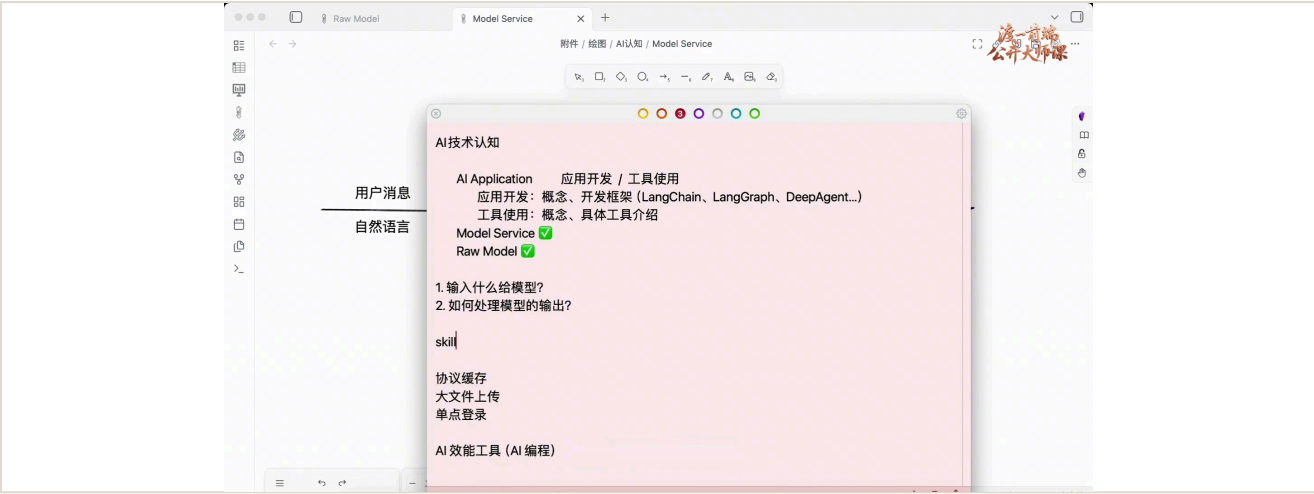
所以说处理是哪一个层面？处理的就是这个层面。我输入什么给模型？好，后处理是什么意思？就是模型给我东西了，我是不是要做一些后续的处理？比方说我之前说过的，在这里边模型服务商一般会做的什么？反 Token 化，还有什么？是不是还有一些合规审查？这些就属于后处理。好，对模型服务商是如此，那么对于AI应用是不是也是如此呢？是一样的。

AI应用考虑的是什么呢？我这里给模型发什么消息。对AI应用来说，那么就可以把整个看作是一个模型，对吧？尽管它里面有一些内部处理，但是对AI应用的话，这里边是黑盒子，我也不管它里面怎么处理的。反正我知道，我给他一个消息，他要回复我一个消息。那我就要想，我要给他传什么消息？对，有的人说：“那这个不是用户填的吗？”是用户填的，用户的消息经过AI应用过后，AI应用有可能给你加消息的。一会儿我们可以给大家演示一下，看有没有给你加消息。也就AI应用那一块，它肯定也会给你加入一些额外的提示词，来一起传给模型。

画面与校读说明

红框圈出模型服务的前处理、自回归和后处理；用户消息与 AI 回复位于两侧。这里展示的核心关系是：应用把消息交给服务，再接收返回内容。

09 · HTTP 类比：看消息、格式与后续动作



课堂画面 · 点击放大

然后后续呢？模型吐出来结果过后，那么AI应用呢会进行后续处理。所以玩来玩去，包括你们学的什么MCP啊、Tools啊、Skills，乱七八糟。不管你们学还是没学，你会发现全都一样，它只能这样玩儿，它玩儿不了别的东西的。所以说你可以把这个逻辑呢，就看成什么呢？有是不是有点像那个HTTP的请求和响应？我给你发什么东西，你给我回什么东西。那比如HTTP里边说，是不是还有什么呢协议缓存，对不对？

玩的是啥？玩的就是我给你发消息，发什么样的格式的消息，你给我响应的时候，就会响应什么样的格式。好，大文件上传，玩的是啥？我给你发的时候，发什么样格式的消息，对吧？发几次？然后你给我响应的时候呢，你会响应什么玩意儿？是不是这样玩的？好，那么 OAuth [术语校读存疑]，或者是单点登录，是不是也是啊？我在规定我给你发什么，然后你给我响应什么。现在对模型也是一样的。

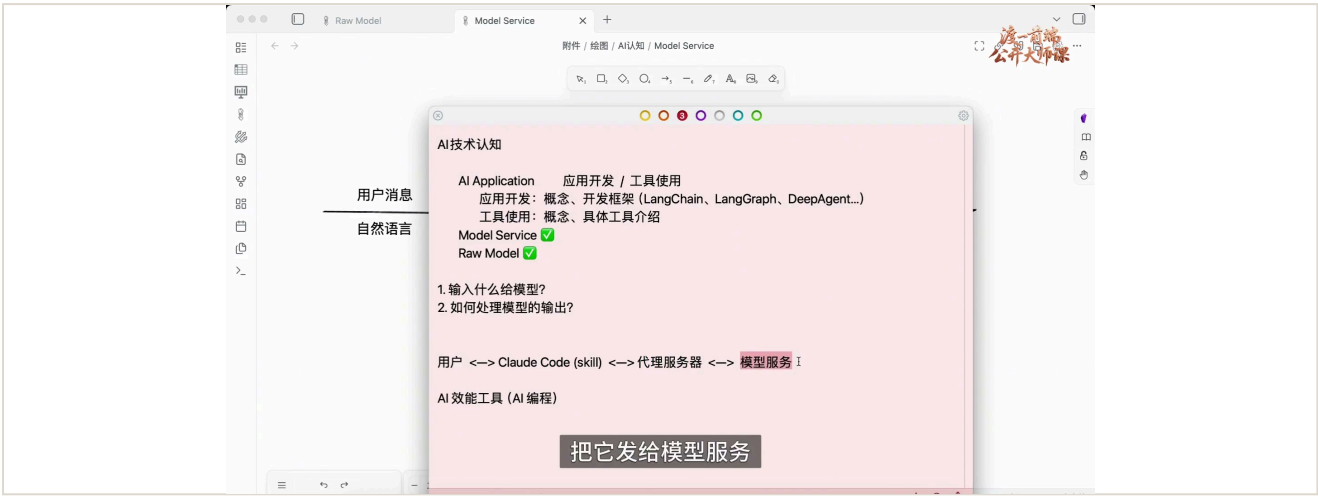
所以说，知道了吧？为什么这两层，在我的眼中是非常简单的，无非天天研究来研究去，就是在玩这个东西。你别管他玩出来多少新概念，那些自媒体，他永远不会告诉你，他也不知道，对吧？他永远也不会告诉你，他玩的东西就那么简单，不然的话，怎么去颠覆全人类的？就不够劲爆了，是吧？但是实际上就这么简单。所以说，同学们以后遇到了一些新的概念，只要是在这两层玩的，你听好了，你只需要做一件事儿

你就可以把它搞清楚。你就看他给模型传了什么东西，然后模型又吐出来什么东西，然后再去研究他接下来又是怎么做的。比方说，我现在就给大家举一个例子：skill，就这个玩意儿，好神秘啊！天天问，天天问。听到这节课之后，你看一下它无比的简单。你只需要知道这个东西，他传了什么给模型，模型又吐出来啥，他接下来又是如何处理的。对吧？就这两个层面：给模型啥，然后如何处理模型的输出。但是呢，我只是以skill为例哈。

画面与校读说明

便签记录 skill、协议缓存、大文件上传、单点登录这些例子。原画面使用“协议缓存”的写法，正文照录；OAuth 的音频辨识不够清楚，已保留术语存疑标记。

10 · 搭一层代理，观察请求和响应



课堂画面 · 点击放大

将来同学们遇到其他概念都可以用类似的方法来分析。所以这节课非常重要，我来教你如何来去理解一个概念。好，怎么做呢？嗯，我们这里呢需要一个代理服务器，看看，现在是用户，用户呢我们假设去使用一个随便吧，比如说 Claude Code 吧，啊当然也可以是别的编程工具，无所谓的。好，那么编程工具这边是不是应该正常的情况下应该是发给一个模型服务商，对不对？他去调这个模型服务，通过一个HTTP请求去调这个模型服务，然后去拿到结果，是这意思吧？

那么模型服务呢，传递结果给Claude Code，那么Claude Code呢，在经过一系列处理过后，把这个结果呢传递给用户，是这意思吧？那么现在呢，你要分析啥？分析这个Claude Code，它不是支持Skill吗？对不对？它是支持Skill的。我现在要分析的是Claude Code，它是把这个Skill是如何跟模型服务之间进行消息传输的。那我这如何来分析呢？，两种方式，一种是抓包，，但是它是HTTPS协议，抓包能不能抓？也可以抓。但是呢，涉及到一些工具，可能有些同学对那些工具都听懂了。

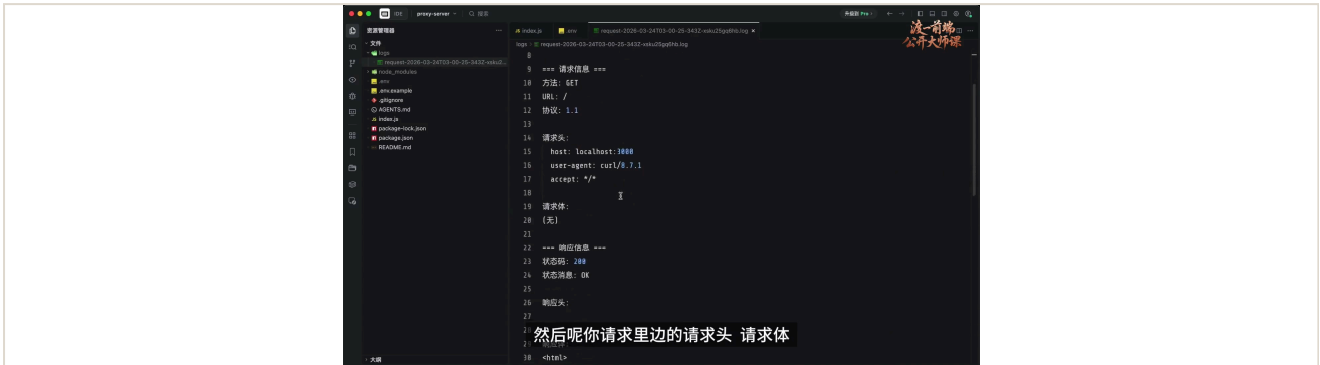
那我这里简单一点，我在这里呢去加一层代理服务器，那就变成了这个逻辑了。我就稍微配置一下Claude Code，让他向模型发消息的时候呢，不要直接发给模型服务，发给我们的代理服务器。我们代理服务器呢进行转发，把它发给模型服务。好，模型服务回复消息过后呢，经过我们的代理服务器，又反馈给这个Claude Code。那么由于这个代理服务器是我们自己写的，我们就可以做一些日志记录，我们就可以看得到他发的消息发的是啥。懂了，意思吧？好，我们先做这么一个工具出来。那么代理服务器呢，同学们如果说不熟悉Node.js的话

你给它提要求，让他帮你写也行，或者是在我这个源码里边直接去下载就行了。好，我这里呢已经写好了哈，我看一下我搁哪儿呢？哦，在这儿，proxy-server。后续的话，我可以把这个东西提供出来，同学们直接拿去用就行了。我这里写的非常简单哈，我也是让AI写的，就一个JS没了，就这个东西。嗯，做了是啥事儿呢？就是启动一个服务器，你给我发个请求，然后我进行转发。

画面与校读说明

便签中的链路是“用户 ↔ Claude Code (skill) ↔ 代理服务器 ↔ 模型服务”。后续代码工程名为 proxy-server，源码入口为 index.js。

11 · 配置 .env, 用 curl 验证代理日志

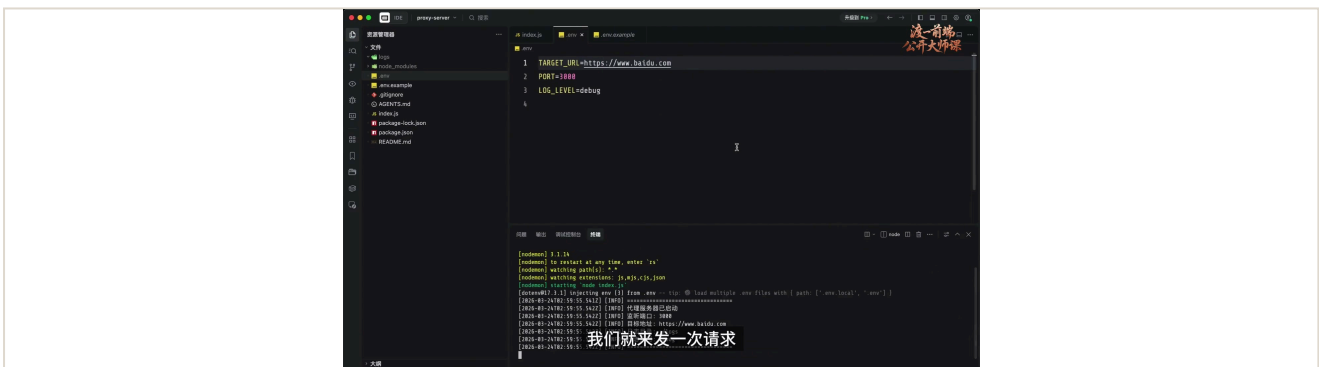


课堂画面 · 点击放大

我们把启动先看一下吧。好，这里运行，`npm run dev`运行出来。好，我这里监听了 3000 端口。好，比方说我去发这么一个请求吧。好，我这里先停止。`.env` 这里边配置一下，就是同学们拿到这个项目过后呢，因为这个我没有加入到Git管理哈。同学们拿到这个项目过后呢，你要把这个`.env.example`，把它改一下名字，或者复制一份变成一个`.env` 文件，这是环境变量。这里至少配置这么几个东西。这个不用管它，都是日志级别。这个是端口，就是你启动那个服务器的端口。

然后这里呢，就是你转发的目标地址。目标地址的话，比方说，我就把转发到随便吧，比方说百度吧。好久没用过百度这个东西了。好，就是所有请求把它发到百度这里。好，来咱们来试一下。现在呢，我们就来发一次请求。我就随便来一个curl，http，发什么发到我们代理，localhost:3000，就直接这样发得了。就这么地吧。走，好，我拿到这个东西，没关系，这个东西是啥吧？，不重要。

好，那你看这边，我这个代理服务器是不是收到请求了？然后呢，他会把它进行日志记录。你看这里，我收到一个请求，我帮你进行转发，转发到这个地址。然后呢，你请求里边的请求头、请求体、响应里边的响应头、响应体，我在这里都会记录下来，是可以便于观察的，对吧？就这么一个代理，其他也没啥。好，那么首先第一步，你要把这个代理服务器呢要准备好。好，接下来这个代理服务器准备好了之后呢，你是不是还要做一件事啊？



环境变量：测试目标、端口与日志级别

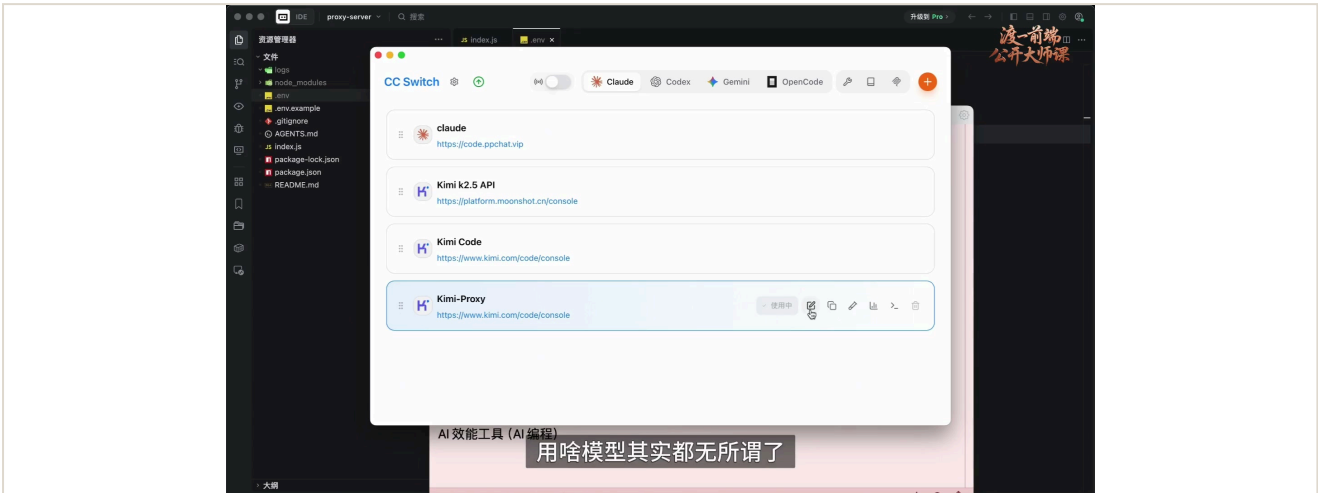
画面与校读说明

日志画面列出请求方法、地址、请求头、请求体，以及响应状态和响应头等信息。配置画面显示如下测试值：

```
TARGET_URL=https://www.baidu.com
PORT=3000
LOG_LEVEL=debug
```

启动命令经音频局部复核为 `npm run dev`；终端测试命令为 `curl http://localhost:3000/`。这些是课堂现有代理项目的操作，不包含该项目的完整源码。

12 · 让 Claude Code 通过代理访问模型



课堂画面 · 点击放大

让你的 AI 编程工具调用模型服务的时候，要去调用你这个代理服务器，不然的话，它会直接去走这个模型服务，那肯定不行。现在几乎所有的 AI 开发工具，它都支持配置，就是你要发到哪个地址，都可以配置的。目前我还没有见过哪个开发工具是不能配置的。那现在问题是，我也不知道你使用的是什么样的 AI 开发工具，你自己去搜一下吧，你的开发工具怎么去配置这个东西啊？我这里呢，开发工具倒是挺多的，关键是我就没必要一个个去讲了。反正开发工具呢，我后面会说。

那我这里就以这个 Claude Code 为例吧。我这里呢配置的方式呢是通过这个 CC Switch。这个有些同学可能知道，有些同学可能不知道，呃不重要，后边我都会说的。总之呢，你们反正自行去配置。我这里呢是启用了 Kimi-Proxy，就是希望这个 Claude Code 呢调用的是我这个代理服务。我这个代理呢，要转发到 Kimi 那边。因为我这里用的不是 Claude Code 官方的模型，我用的是 Kimi 的模型。用啥模型其实都无所谓了，这个我后边会说。

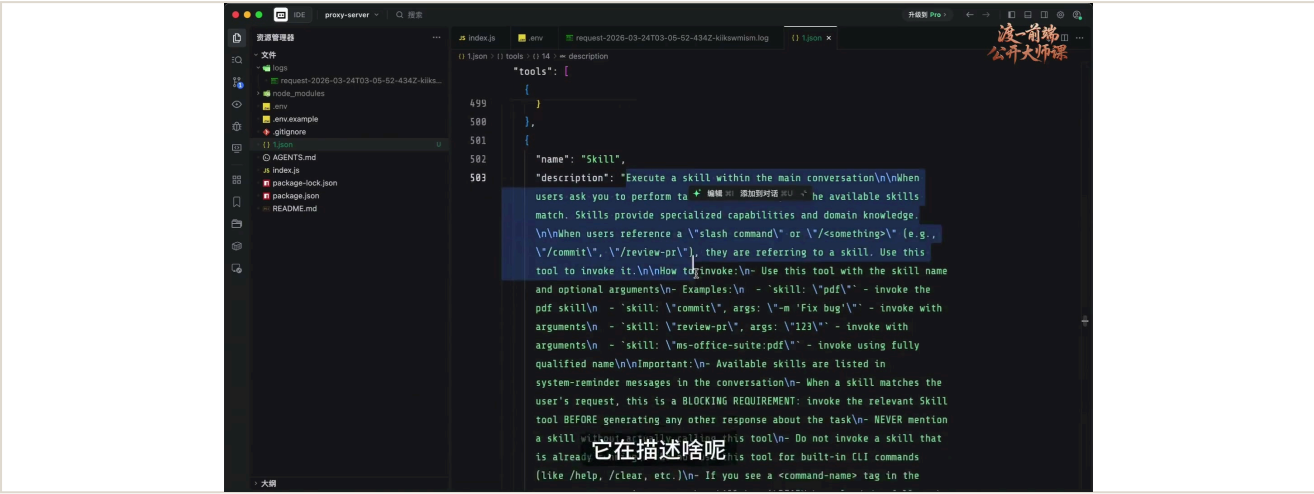
你经常的问哪个模型好，哎呀，我哇，这种事情我以后会说。好，这里就编辑一下就可以了。你看，我编辑的就是这个地址，请求的是啥？我们的代理服务器，看到没？放大一点。其他的呢配置呢都没有什么变化。然后呢，我就把这个东西启用。目前就启用的是个代理。那么后续的 Claude Code 呢，他就走这个代理服务了。那么其实在这里吧，也可以看到，它其实就是给你生成这么一个配置。我靠，我的 Key 都暴露了，暴露就暴露吧。

好，然后把这个启用过后呢，现在我们就随便找个工程，就这个工程吧，然后进入 Claude Code。你先你看，现在我们的这个线就通了，是吧？它走的是啥代理服务器？代理服务器让它转发到模型服务。当然，我们这个代理服务器这里还要设置一下，就不能是百度了，不能转发到百度，要转发到 Kimi 这边。好，然后重新来，启动。好，接下来我们就进入 Claude Code，看一下给它发的是啥东西。走进去。好，进去过后呢，你看我们的代理就生成了一条日志了。

画面与校读说明

CC Switch 中选中了 Kimi-Proxy。应用端的请求地址应指向本地代理，代理的 TARGET_URL 应指向实际模型服务；百度地址仅用于前一步测试。选用的截图展示配置列表，未采用含凭据的配置展开画面。

13 · 还没输入问题，应用已经发送了工具描述



课堂画面 · 点击放大

你可以看到，Claude Code，他首先往这个模型这里发了这么一条消息，请求的是这个地址，请求头乱七八糟一大堆。然后看一下请求体，请求体传递的是一个json格式。首先，同学们看一下，我们现在有没有向那个Claude Code在这里边写任何消息啊？没有是吧？说明啥？看一下之前这张图，说明我们上边的用户，不管你填没填消息

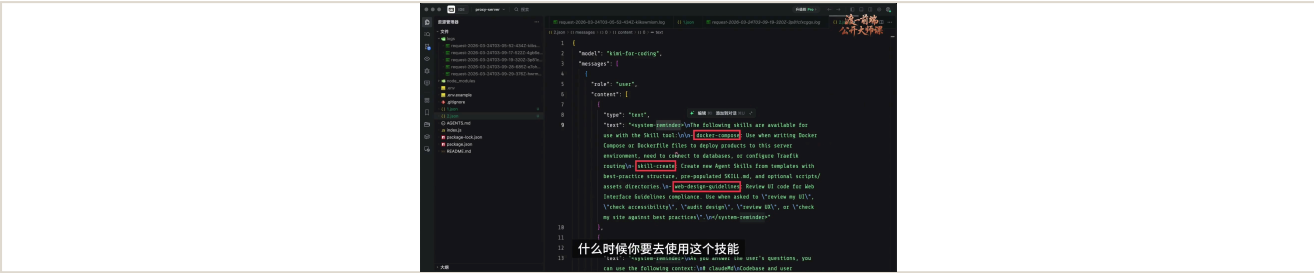
都要经过这个AI Application，就是AI应用给你进行封装一层，它有可能给你加入额外的消息，有可能你啥消息都没写，也会向模型给他发送一段消息。那我们可以简单的看一下，这个消息的内容是啥。好，走，我们把这个复制一份儿。好，然后这里去建立一个JSON，走，保存一下，给他发送一个消息，简单看一下吧，这里边有啥东西。好，首先呢，给他传递了一个用户的消息，就是一个测试字符，他测试一下这个模型通不通。

然后呢，给他传递了一下这个tools，这个tools的概念呢，我们后续会讲。好，在这些tools里边呢，我们可以看到，有这么一个tools叫做skill，它在描述啥呢？它在告诉AI模型，什么叫做技能，技能的是什么意思，技能的含义是啥？然后呢，同时也约定了AI模型说，如果说你要将来要使用一个技能，你要给我传递什么东西，肯定是要有技能的名字是吧？

画面与校读说明

请求 JSON 的 tools 列表中有 name 为 Skill 的条目，description 说明技能的使用时机、技能名称和可选参数。这里的 Skill 是工具名，技能清单则在后续提示词里列出。

14 · “你是谁” 的答案，来自应用添加的提示词



课堂画面 · 点击放大

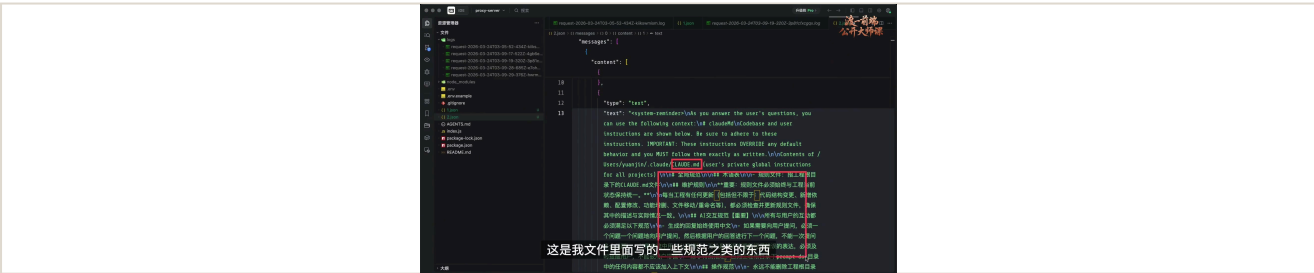
还有参数怎么传递，它就在描述这个。那么这个传递过去，AI模型是不是就知道了技能是啥？然后呢，如果说我要使用技能，应该怎么来回复消息，对吧？我们现在啥都没做。那么这个AI应用他就把这个消息传过去了。好，接下来我们来写个消息吧。你是谁？好，看他的回复。好，他说他是Claude Code，，是一个什么CLI工具。但是这个信息是谁回复的？那肯定是模型回复的呀。

但是我明明用的是Kimi 模型，为什么他会回复我这个消息呢？是不是很奇怪？那你看一下他的请求，他给加了一大堆请求，没关系，我就随便看一个。好，我们把这个请求体，还是来复制一下，便于同学们看得清楚一点。我们这里乱七八糟一大堆，好，来看一下。我们在这里去搜索一下，你是谁？就是我们的题的问题，你看，这是给AI发的消息，你是谁？但是AI只收到这个消息了吗？并不止，他前面还有消息

你看这一段，这一段是啥呢？这一段是我们工程里边的那个CLAUDE.md这个文件，看到没？是不是这个文件？之前是Claude 官方怎么说的，就是这个文件，是不是在每一次消息发送的时候都会带过去？你看是不是有带过去？这是我文件里面写的一些规范之类的东西，它给带过去了。那么除此之外，还有没有带东西过去呢？还有，你看这个System Reminder，就是系统提醒，这就是Claude Code 这个工具给你加的消息。

它这里边说，首先告诉了AI，我有哪些技能是可以用的。我有这么一个docker-compose的技能，还有这个skill-create的技能，就是我写的那个技能的MD是可以用的。你看技能是怎么带过去的，技能就是提示词，给它带过去的。还有什么技能？还有这个web-design-guidelines。每个技能后面有描述，看后面都是描述。告诉AI，我有哪些技能，什么时候你要去使用那个技能。这个描述就是告诉AI，什么时候要去使用那个技能。

至于说怎么去使用技能，所以之前告诉过AI了，要返回什么样的消息格式来去使用技能。所以技能是不是非常简单，就是个提示词。好，除了这个消息之外，你看下边的系统提示词里面还有。这里又得告诉AI你是谁，你是一个可交互的 CLI 工具。是刚才我问他你是谁的时候，他是从这边找到答案的。这是来自于Claude Code的工具给AI喂的提示词。

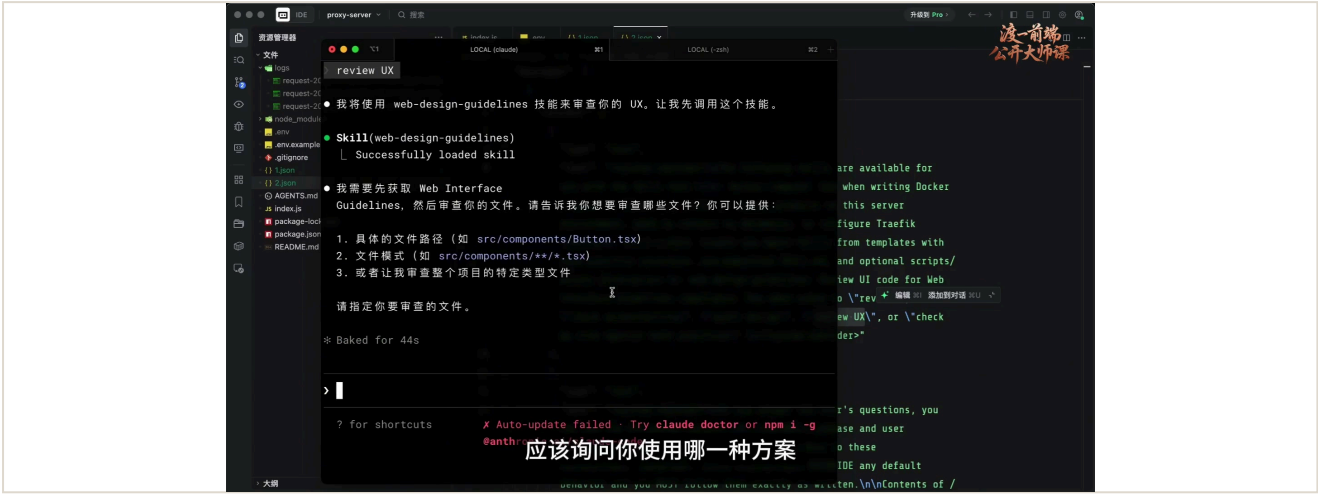


CLAUDE.md 内容进入请求上下文

画面与校读说明

请求标明 model 为 kimi-for-coding；system-reminder 列出 docker-compose、skill-create、web-design-guidelines 及各自描述。另一个画面显示 CLAUDE.md 文本被送进上下文；画面中的路径属于用户级全局规则，讲解中也提到工程规则。这里记录本次演示实际观察到的请求，不推断所有版本或所有请求都相同。

15 · 触发一次技能，再去查真实请求



课堂画面 · 点击放大

再回过头来看一下，我们玩的 Skills〔术语校读存疑〕，包括什么那些工具在干啥，就在控制输入什么给模型。那么接来看第二个，它如何来处理模型的输出的。咱们又来一次，把这删了。我举个例子哈，比方说，他怎么来调用技能的。比方说这个技能吧，他说，当你去使用这些语句的时候呢，他就会去调用这个技能。

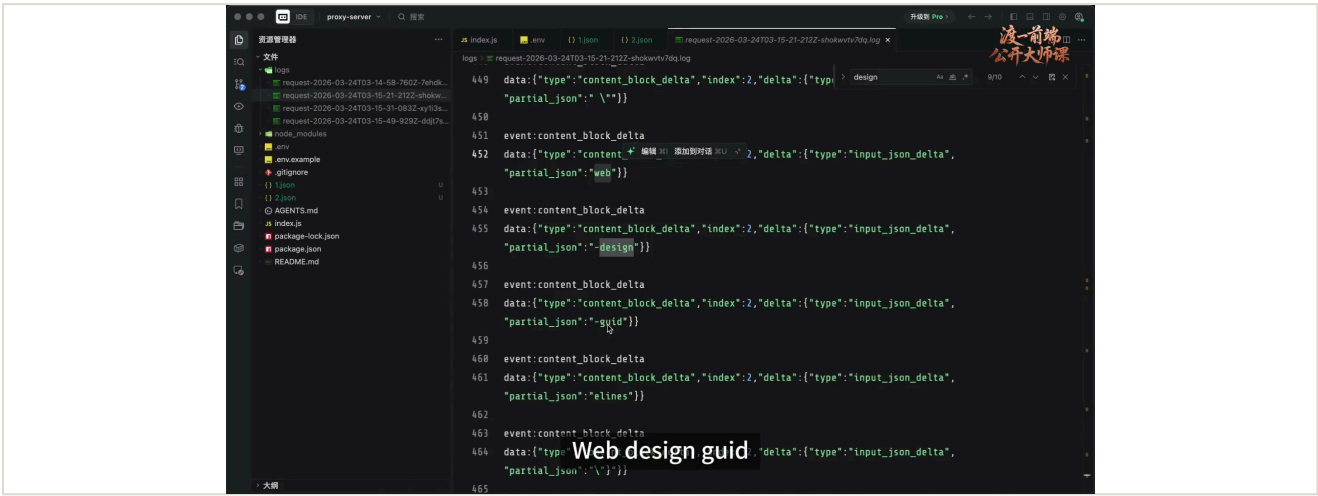
使用这个语句，来看一下。好，你看他现在调用技能是吧？他已经成功的加载了技能。我们看一下他这个过程是怎么样的。好，那么这里他在询问你，他肯定是读过这个技能文档过后，他才能知道应该询问你使用哪一种方案。我现在不去选择了，我们来看一下发那个请求是怎么发送的。好，来看一下，咱们的请求是那个review是吧？review UX。好，你看，都是我们用户发的请求，文本是review UX。

现在首先问大家：AI，他知不知道有哪些技能？现在模型知不知道？模型它是知道的。之前是不是看过了？给他告诉他的对吧？我们系统里面有哪些技能？他是知道的。而且他也知道什么时候要去使用技能，对吧？那么现在用户发了这个消息，他是不是应该使用技能了？那你看一下AI给我们的是什么样的回复啊？我们来搜这个 design。因为我们那个技能的名字叫做web-design，是吧？来搜一下 design。

画面与校读说明

终端实际输入是 review UX，不是转写里的“review邮箱”。返回显示 Skill(web-design-guidelines) 和 Successfully loaded skill，随后要求用户指定文件路径、文件模式，或审查整个项目中的特定类型文件。

16 · SSE 流里的自然语言与工具调用 JSON



课堂画面 · 点击放大

看这里，这是AI的响应，响应呢，它也是通过HTTP来响应的。为什么是这种格式呢？同学们，你们没见过这种格式？这个东西格式叫啥呀？这个格式呢，叫做SSE。它是流式传输，它每次传输一段，你看，每次传输一段。我将使用web-design-guidelines，看到没？它这里对应到哪儿？是不是对应到这儿？，它只是一个回复而已。那么现在真正的技能调用，是不是要满足某一种格式？

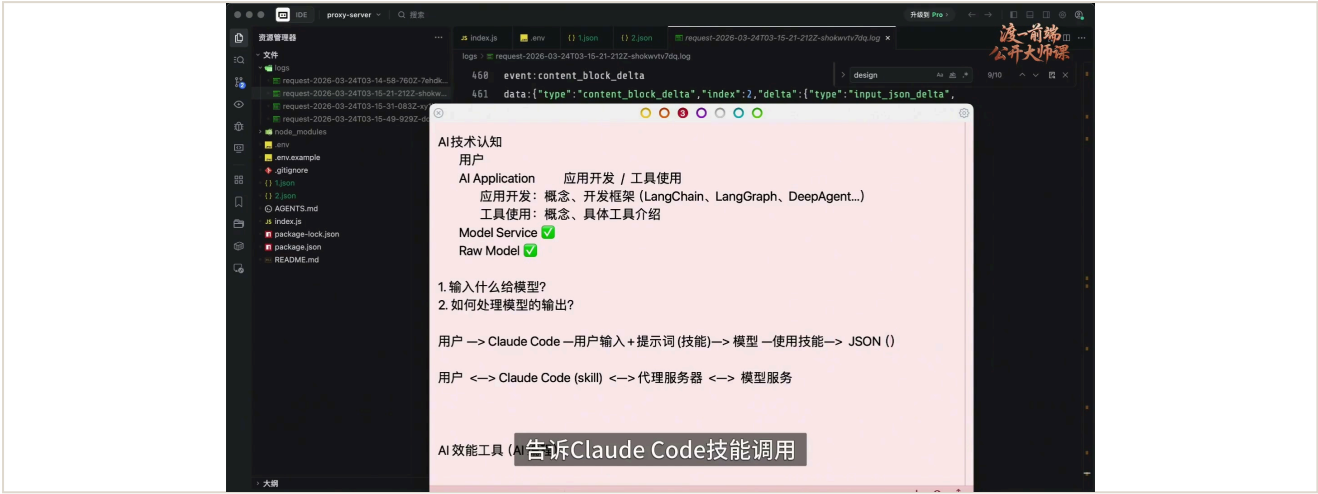
之前是不是看过，见过某一种格式的？就是当时我说，就在最开始的时候，是不是告诉过AI，当你要使用技能的时候，要给我返回一种JSON 格式，是吧？好，那你看一下，它JSON 格式在哪里调用的，就在这里。它继续响应。前面的是自然语言响应，完了过后，接下来它响应的一个是，tool_use使用工具，什么工具呢？使用了一个技能工具。哎，这个东西是还记得不？不好意思，刚才把那个日志已经删了，看不到了。

刚才日志最开始告诉模型有哪些工具。其中一个工具名字叫技能。当你要使用技能的时候，你要给我传一个JSON，看到没？，因为它是流式的，它隔开了。你看这个JSON的大括号，看到没？然后呢，使用的是一个skill，然后后边肯定有skill的名字在哪里呢？在这里，web-design-guidelines，这是技能的名字，拼接好了。那这样子一来，这个Claude Code是不是就可以拿到这个模型的响应？

画面与校读说明

SSE 中反复出现 event: content_block_delta; data 对象的 delta.type 为 input_json_delta, partial_json 逐段出现 web、-design、-guid、elines 等片段。片段拼接后形成技能名称。这些事件名与字段名来自所选画面。

17 · 第一轮：封装输入，模型决定是否调用技能



课堂画面 · 点击放大

来看一下流程。用户有一个输入，发给谁？发给Claude Code了。Claude Code接收到的输入加了一大堆系统提示词，发给什么模型？那么这里除了有用户的输入之外，还有什么？还有就是各种提示词。那么这个提示词里面就包含了技能，我有哪些技能？你到时候要使用技能的时候，你再给我回复什么。这些一起就传给模型了。

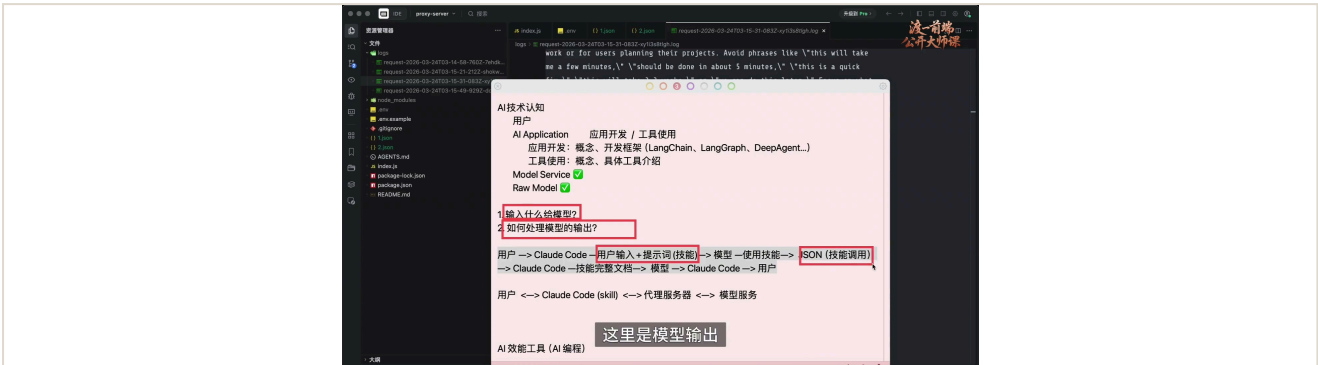
好，模型这个时候叫判断用户的输入，模型拿到的是输入，拿到什么输入，这个东西加上这个东西，对吧？那么这个时候模型就要进行判定了。那么这个时候到哪儿了？说到这儿了，到这个模型，模型最底层的模型层了。那么到这儿了过后，是不是还要判断我要不要使用技能？如果说不使用技能的话，比方说一开始我问的是你是谁，他跟技能没关系，他就正常输出就完事儿了。

如果要使用技能，决定使用之后，他会返回来是一个什么？返回来是一个JSON。这个JSON要告诉什么？告诉Claude Code技能调用，我要使用某一个技能，传给谁了？又传给Claude Code了。看到没，就是在这里进行响应的。我要使用这个web-design-guidelines这个技能。好，那么接下来Claude Code拿到这个响应过后，他又是怎么处理的呢？他不会直接给用户什么消息。

画面与校读说明

便签按顺序补出用户输入、Claude Code 添加提示词、模型判断使用技能、返回 JSON（技能调用），再交给 Claude Code 的链路。

18 · 第二轮：读取技能全文，继续调用模型



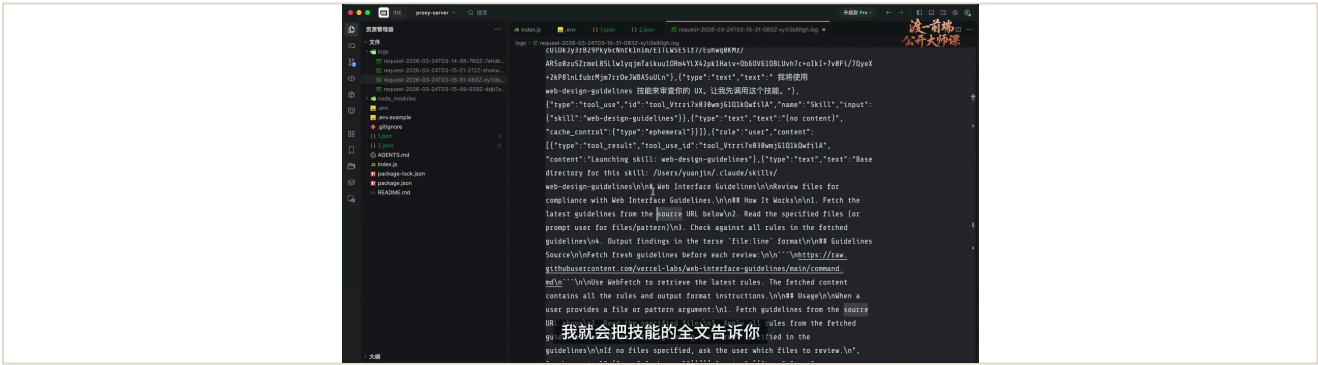
课堂画面 · 点击放大

它会进一步去读这个技能文档，这个能不能读？它是个本地程序，肯定能够把个技能文档读出来呀。你看一下有没有读到这个技能文档，咱们来搜一下这个技能。这个技能，注意，看一下，可以进去。这个技能在这儿，我们就随便搜一句话吧，比如说这一句话，这是我们的一个技能，看到没？web-design-guidelines，技能文档里边复制一下

然后到这边来搜索一下。你看，现在要使用技能了，看到没？那么这个时候，Claude Code就会把技能的全文读出来。一开始有没有给他全文？没有。一开始只是告诉，我有哪些技能。你要使用的时候，就会把技能的全文告诉你。你看这里，这就是技能的全文，就把文档里边所有东西全部读出来，把文件内容读出来。读文件不复杂吧？然后一起扔给AI，让AI去进行后续的处理。那也就是说，Claude Code会怎么样呢？

会把技能完整文档又传给模型，然后模型去进行后续处理，然后最后呢，模型的东西又传给Claude Code，然后又传给用户。那么于是呢，用户就看到了这个提示。好，捋一下这个流程。整个流程，你看一下哪些是在给模型输入，这里是不是给模型输入？喂提示词，哪些是在处理输出？

哪些在处理输出？这里是模型输出，然后Claude Code做后续处理。什么后续处理呢？又重新去调用模型，给它完整文档。然后这是模型的输出，输出给Claude Code，Claude Code再给用户。看到没？这就是个完整的闭环。你通过这个代理服务器就可以非常清楚的看到，它传了什么东西给模型，模型又吐出来啥，它又是如何进行后续处理的。

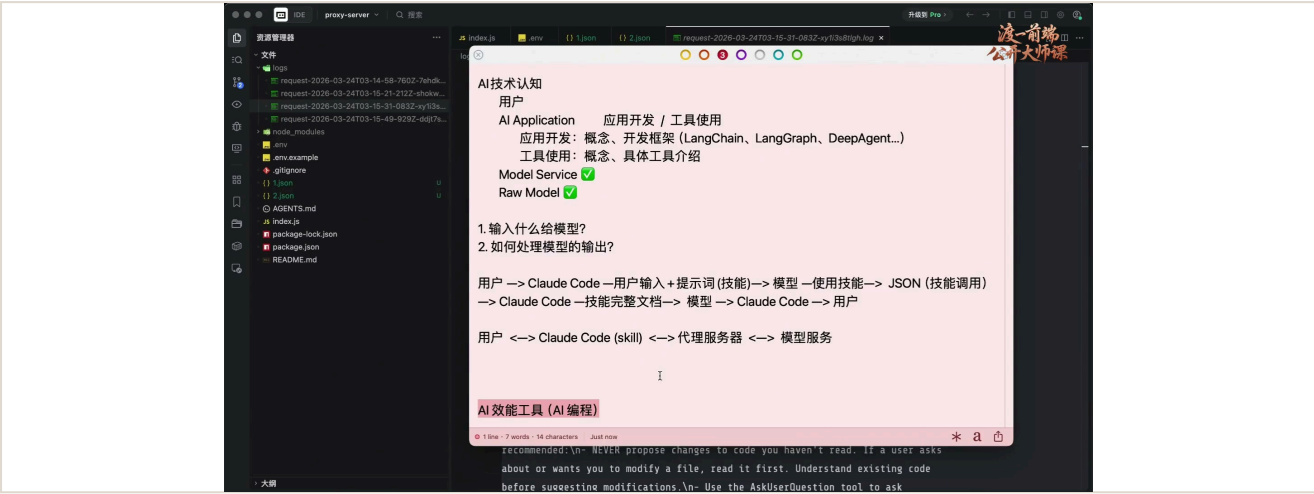


下一轮请求包含技能文档正文

画面与校读说明

红框分别标出给模型输入与处理模型输出的位置。补充日志画面能看到 tool_use、Skill、input.skill、tool_result，以及 Web Interface Guidelines 文档正文；这是第二轮请求包含技能内容的直接证据。

19 · 把观察方法迁移到下一个新概念



课堂画面 · 点击放大

但我再次重申，我讲的是 Skill 吗？不是，我后边讲那个AI效能工具的时候，我还会讲Skill。我现在讲的不是Skill，我是通过这个Skill这个例子，告诉同学们：以后遇到了那些乱七八糟的各种新概念的时候，你怎么知道这个概念到底是啥？没有什么新奇的，包括现在什么OpenClaw [名称校读存疑]，我估计有些同学可能看到这个视频的时候，那个东西都已经消失了。不过它消不消是不重要。总之后看到新概念，你就研究这两个东西就行了。它到底输入什么给模型，又得到什么样的一个输出结果。

因为说来说去，整个模型到现在几年了，它就没有变化，就是输入输出，它玩不了别的东西，别的东西都是这两层玩出来的。听懂了吧？好，那咱们AI技术认知这一块就到这儿了。有同学说：“那你只解释了几个呢，还有好多东西没解释。”我哥们儿，你要让我去解释这个，那你想多了，没必要，没必要。

如果听到现在还需要逐个解释概念，那真的是想多了。不过这些概念要不要解释呢？要解释，我后边在那个编程AI效能工具里边去解释。你有了现在这种认识过后，后边我说一个概念，我都不用去给你看这个具体的这个传输的内容了，因为这些东西我都看过的。我一说你就明白了。我就告诉你它传给模型啥，然后模型它吐出来啥，它又怎么进行后续操作的，就完事儿了。后边还会对各种概念进行对比。所以说，咱们学任何东西，抓住核心，不要去管那些细枝末节。

画面与校读说明

画面保留完整调用链与 AI 应用层笔记。讲者在这一段再次说明，Skills 是用来展示分析方法的例子，随后会在工具课程中继续讲解具体概念。

20 · 抓住核心与后续课程安排



课堂画面 · 点击放大

不然容易陷入知识的海洋，出不来了。就像我们之前学前端一样，前端最火的那几年，是不是也是各种概念层出不穷啊？你看我们的学员care吗，并不care，无所谓。首先，你干啥玩意儿？你玩着玩着，还能玩出个啥玩意儿？都是新瓶装旧酒。而且很多新玩意儿也活不过很长的时间。抓住核心。好，那么AI技术认知这一块就到此结束了。那么从下节课开始呢，我们要讲的AI相关工具，这里给同学们提示一下。后续的东西呢，就不再免费发出来了哈，就在我们的付费课里边了

因为我全部发出来吧，咱们付费学员得动手。说后边的东西呢，我会放到哪儿呢？会放到咱们高薪课。因为高薪课、架构课呢，它都有AI内容，但是层面不一样。架构课呢是主要是集中在我如何去开发一个AI应用，而高薪课呢主要集中在我去如何使用AI应用，对吧？一个是使用层面，它是属于业务开发的；另一个是架构层面，它是属于架构师的，对吧？

我去开发一个效能工具，提升你的效率，降低整个团队的成本，那是属于架构级别的。我们这个开发AI应用的级别的东西呢，在架构课里边已经有了哈。那我们这个AI效能工具的使用层面的东西呢，在我们的高薪课里边。你具体需要什么东西，你自己来找咱们了解。再说一次，找咱们的方式，在咱们账号主页点击头像进入账号主页，根据提示来找咱们领取课程就可以了。

画面与校读说明

课程介绍叠放在完整流程笔记上，结尾区分 AI 应用开发与 AI 工具使用两条学习路线。课程获取方式保留原课语境。

来源：用户提供的《4.AI 本质.mp4》。正文依照课堂顺序轻度整理；课程中的产品、价格、工具能力和讲者判断保留录制时语境，不作为当前产品规格。

本地 Qwen3-ASR 0.6B INT8 转写，结合画面核对话术；原始音频块保存在 source-materials。时间仅为音频块粒度，不是逐词对齐；未进行逐字人工听校，可能仍有识别误差。已对全部 69 个音频块交界和 5 处疑点做本地音频重识别；这不等于逐字人工听校。实际模型 Token 用量与费用未由运行环境提供。